西南交通大学
Southwest Jiaotong University

《算法分析与设计B》

# 蛮力法

计算机与人工智能学院
李晓航
xhli_scce@swjtu.edu.cn

Last: 2024-03

# 一、弯力法概述

- 1.1 弯力法定义
- 1.2 弯力法求解框架
- 1.3 弯力法的作用
- 1.4 弯力法的例子

# 1.1 蛮力法定义

- 蛮力法定义
  - 基于问题的描述和所涉及的概念、定义直接求解，逐一列举并处理问题所涉及的所有情形，而后得到问题的答案的方法。
  - 蛮力法是一种简单直接的设计策略，也叫暴力法、枚举法或穷举法。
- 蛮力法的特点
  - 优点：逻辑清晰，通常是解决问题的最直接的算法。
  - 缺点：效率低下。
- 蛮力法是很多算法的基础，对蛮力法加以优化，有可能得到效率更高的算法
  - 但对蛮力法进行优化只能减少算法复杂性的系数，不会改变复杂性的数量级。

# 1.2 蛮力法求解框架

- 蛮力法的基本技术—扫描技术
  - 即采用一定的策略将待求解问题的所有可能的解依次处理一次，从而找出问题的解。
  - 基本的扫描技术：线性表的遍历、集合的遍历、树的遍历（DFS或BFS）、图的遍历（DFS或BFS）等。
- 蛮力法的求解框架

```
BruteForce()
{
    n = 0;
    for(k = 区间下限; k <= 区间上限; k++)
        if(约束条件)
        {   找到一个解;
            n++;
        }
    return (汇总的解);
}
```

注：算法设计中的蛮力法通常指回溯法，用于对问题的解空间树（子集树或排列树）按DFS方式进行搜索，而不是用 for 循环。

# 1.3 蛮力法的作用

- 虽然巧妙和高效的算法很少来自于蛮力法，但蛮力法仍是一种重要的算法设计技术
  - 1、理论上蛮力法可以解决可计算领域的各种问题。
  - 2、蛮力法经常用来解决一些较小规模的问题。
  - 3、蛮力法可以作为某类问题时间性能的底限，来衡量同样问题的更高效算法。

# 1.4 蛮力法的例子

- 百钱百鸡问题
  - 一只公鸡值钱 5 元，一只母鸡值钱 3 元，三只小鸡值钱 1 元。如果希望花 100 元钱买100只鸡，问如何买？
- 算法设计
  - 循环变量：设 x，y，z 分别为公鸡、母鸡、小鸡的数量。
  - 循环范围：由题意给定共 100 钱要买百鸡，若全买公鸡最多买 100/5=20 只，显然 x 的取值范围 1~20 之间；同理，y 的取值范围在1~33 之间，z 的取值范围在 1~100 之间。
  - 约束条件： x+y+z=100 且 5*x+3*y+z/3=100。

# 蛮力法的例子（续）

- 算法1：

```
main( )
{ int x,y,z;
   for(x=1;x<=20;x=x+1)
      for(y=1;y<=34;y=y+1)
         for(z=1;z<=100;z=z+1)
            if( (100=x+y+z) and (100=5*x+3*y+z/3) )
            {
               print("the cock number is",x) ;
               print("the hen number is", y) ;
               print("the chick number is ",z);
            }
}
```

此算法需要枚举尝试20*34*100=68000次。

算法的效率显然太低！

# 蛮力法的例子（续）

- 改进的算法

```
main(  )
{  int x,y,z;
   for(x=1;x<=20;x=x+1)
      for(y=1;y<=33;y=y+1)
      {
        z=100-x-y;
        if( (z mod 3=0) and (5*x+3*y+z/3=100) )
          {print("the cock number is",x)；
           print("the hen number is", y)；
           print("the chick number is ",z);}
       }
}
```

以上算法只需枚举尝试20*33=660次（二重循环）。

实现时约束条件限定Z能被3整除时，才判断“5*x+3*y+z/3=100”。这样进一步提高了算法效率。

# 二、排序问题

- 2.1 选择排序
- 2.2 冒泡排序

# 2.1 选择排序

- 基本思想
  - 1、扫描整个元素列表，找最小元素，并与第 1 个元素交换位置；
  - 2、从第 2 个元素开始找余下元素的最小元素，与第 2 个元素交换；
  - 3、如此继续，直到全部元素排好序。一共 找 n-1 遍。

| 1 | 2 | 3 | 4 | 5 | 6 | $n=7$ |
|---|---|---|---|---|---|---|
| 89 | 45 | 68 | 90 | 29 | 34 | 17 |
| 17 | 45 | 68 | 90 | 29 | 34 | 89 |
| 17 | 29 | 68 | 90 | 45 | 34 | 89 |
| 17 | 29 | 34 | 90 | 45 | 68 | 89 |
| 17 | 29 | 34 | 45 | 90 | 68 | 89 |
| 17 | 29 | 34 | 45 | 68 | 90 | 89 |
| 17 | 29 | 34 | 45 | 68 | 89 | 90 |

# 选择排序（续）

- 选择排序的第 i 趟排序过程
  - 从第 i 个记录开始扫描序列，在后续的 n-i+1（1≤i≤n-1）个记录中找到最小的元素，并和第 i 个元素交换作为有序序列的第 i 个记录。
- 选择排序具备在位性，不具备稳定性。

```
SelectionSort(A[1...n])
{
 for i ← 1 to n − 1 do
     min ← i
     for j ← i + 1 to n do
         if (A[j] < A[min]) min ← j
     A[i] ↔ A[min]
}
```

该算法的基本语句是内层循环体中的比较语句，其复杂度为：

$$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}1=\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}=O(n^2)$$

# 2.2 冒泡排序

- 基本思想
  - 1、比较两个相邻元素，若为逆序，彼此交换位置。如此多次，最大元素上升到最后一个位置（第 1 轮）。
  - 2、第 2 轮将第 2 大元素上升到倒数第二个位置。
  - 3、如此 n-1 轮，排序完毕。

| 1 | 2 | 3 | 4 | 5 | 6 | $n=7$ | |
|---|---|---|---|---|---|---|---|
| 89 ↔ | 45 | 68 | 90 | 29 | 34 | 17 | 第1轮 |
| 45 | 89 ↔ | 68 | 90 | 29 | 34 | 17 | |
| 45 | 68 | 89 ↔ | 90 | 29 | 34 | 17 | |
| 45 | 68 | 89 | 90 ↔ | 29 | 34 | 17 | |
| 45 | 68 | 89 | 29 | 90 ↔ | 34 | 17 | |
| 45 | 68 | 89 | 29 | 34 | 90 ↔ | 17 | |
| 45 | 68 | 89 | 29 | 34 | 17 | 90 | |
| 第$i=2$轮开始 | | | | | | | |
| 45 ↔ | 68 ↔ | 89 ↔ | 29 | 34 | 17 | 90 | 第2轮 |
| 45 | 68 | 29 | 89 ↔ | 34 | 17 | 90 | |
| 45 | 68 | 29 | 34 | 89 ↔ | 17 | 90 | |
| 45 | 68 | 29 | 34 | 17 | 89 | 90 | |
| ..... | | | | | | | |

# 冒泡排序（续）

```
void BubbleSort( int r[1..n ] )
{
    for (i=1; i<=n-1; i++)
        for (j=1; j<=n-i; j++)
            if (r[j]>r[j+1])  r[j]←→r[j+1];
 }
```

该算法的基本语句是内层循环体中的比较语句，其算法复杂度为

$$\sum_{i=1}^{n-1}\sum_{j=1}^{n-i}1=\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}=O(n^2)$$

# 冒泡排序算法的改进

- 改进1：在一趟冒泡排序过程中，如果有多个记录交换到最终位置，则下一趟冒泡排序将不处理这些记录。
- 改进2：在一趟冒泡排序过程中，如果没有记录相交换，那么表明这个数组已经有序，算法将终止。

```
void BubbleSort（int r[1..n ]）{
   exchange=n;                    //第一趟冒泡排序的范围是r[1]到r[n]
   while（exchange）   {
       bound=exchange;
       exchange=0；
        for（j=1; j<bound; j++）      //一趟冒泡排序
           if（r[j] > r[j+1]）{
              r[j]←→r[j+1];
              exchange=j;          //记录每一次发生记录交换的位置
           }
    }   }
```

# 冒泡排序（续）

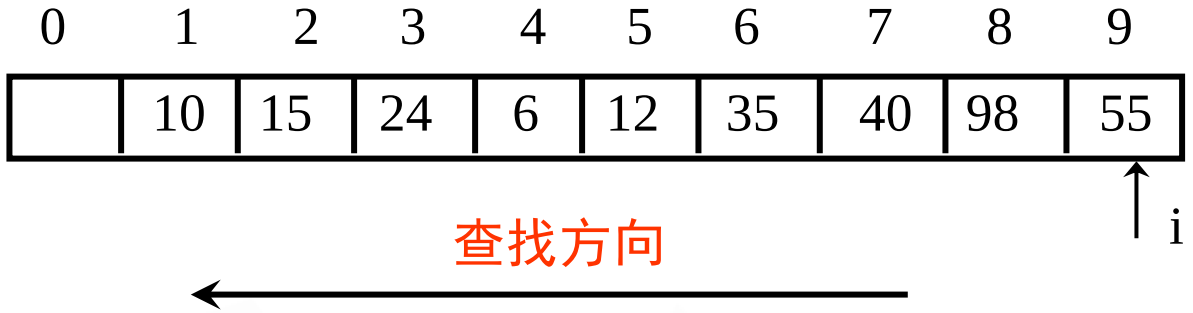
- 冒泡排序具备在位性，也具备稳定性。

# 三、查找问题

- 3.1 顺序查找
- 3.2 字符串匹配

# 3.1 顺序查找

- 基本思想
  - 从表的一端向另一端逐个将元素与给定值进行比较，若相等，则查找成功，给出该元素在表中的位置；
  - 若整个表检测完仍未找到与给定值相等的元素，则查找失败，给出失败信息。



# 顺序查找（续）

```
int SeqSearch1(int r[1..n], int k)    //数组r[1] ~ r[n]存放查找集合
{
    int i=n;
    while(i>0 && r[i]!=k)    //从后往前查找
        i--;
    return i;
}
```

本算法的关键操作语句是 i>0 和 r[i]!=k，其执行次数为:

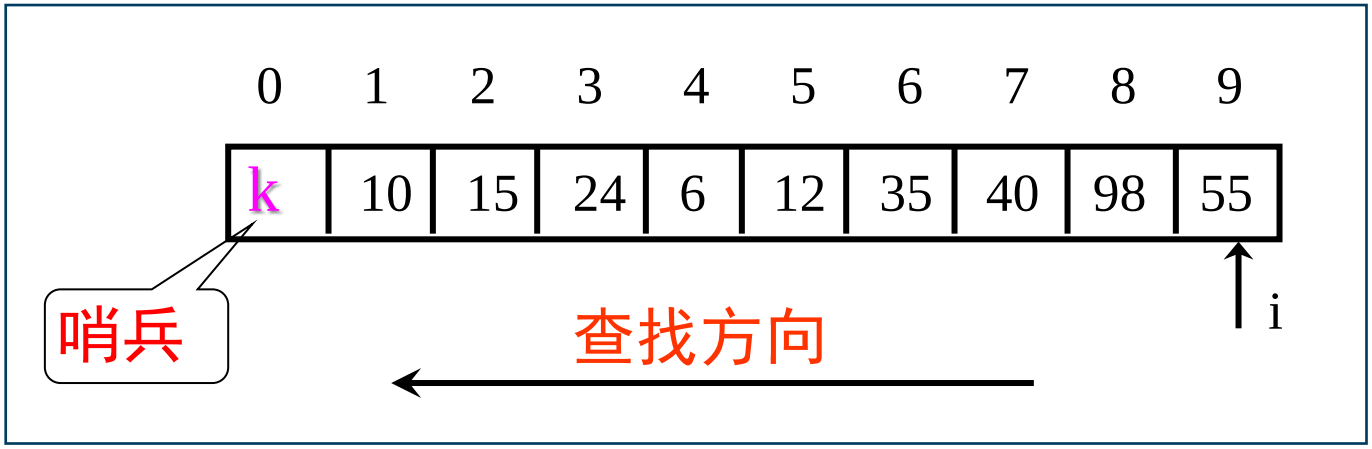
$$\sum_{i=1}^{n} p_i b_i + \sum_{i=1}^{n} p_i c_i = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) + \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = n+1 = O(n)$$

$b_i$，$c_i$：指当所查找的数位于数组的第 i 个单元时，基本语句的执行次数；

$p_i$：假设位于哪里的几率均等，为 1/n 。

# 改进的顺序查找

- 将待查值放在查找方向的尽头处，作为“哨兵”，免去了在查找过程中每一次比较后都要判断查找位置是否越界，从而提高了查找速度。



# 改进的顺序查找（续）

```
int SeqSearch2(int r[ 1..n ],  int k)
   {
     int i=n;
     r[0]=k;                  //保存哨兵
     while (r[i]!=k)  //不再判断 i
         i --;
     return i;
}
```

一般来说，用蛮力法设计的算法经过适度的努力后，都可以对算法的第一个版本进行一定程度的改良，改进其时间性能。但只能减少系数，而数量级不会改变。

算法的基本语句是 r[i]!=k，其执行次数为:

$$\sum_{i=1}^{n} p_i c_i = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2} = O(n)$$

数量级相同，系数相差一半

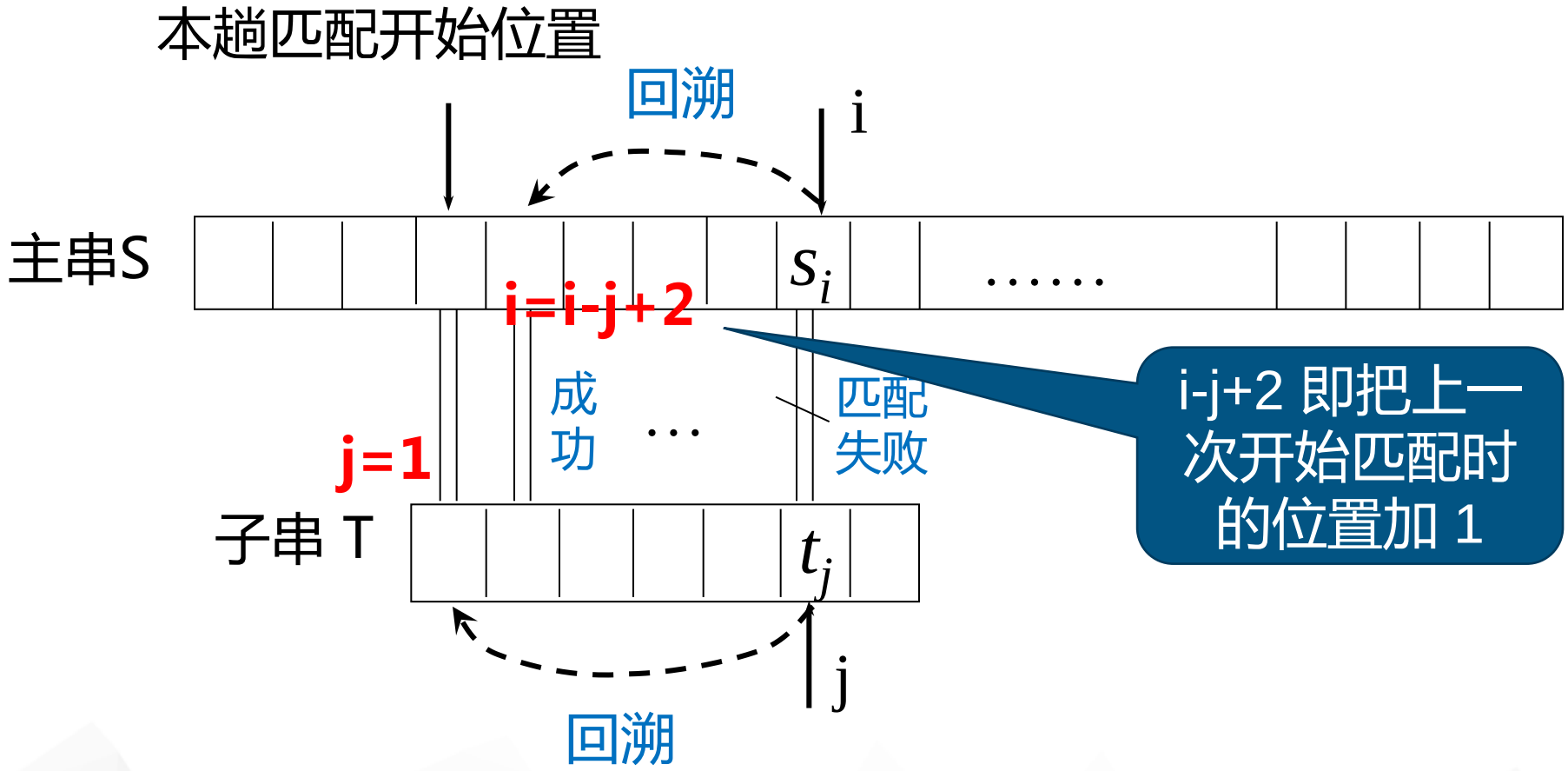
# 3.2 字符串匹配

- 字符串匹配问题
  - 给定两个串 S=“$s_1s_2…s_n$“和 T=”$t_1t_2…t_m$“，在主串 S 中查找子串 T 的过程称为字符串匹配，也称模式匹配。
- 串匹配问题的特征
  - 1、算法一次执行时间不容忽视：问题规模 n 可能很大，常常需要在大量信息中进行匹配；
  - 2、算法改进所取得的积累效益不容忽视：串匹配操作经常被调用，执行频率高。

# 字符串匹配的原理

- 将子串 T 对准主串 S 的开始位置，从左向右逐一检查二者的对应字符
  - 如果相同，则检查下一字符；
  - 如果不同，则将子串 T 的起始位置重新设成开始位置（相当于对应的匹配位置又重新开始）。
- 如果最终匹配成功，则返回最左端字符在主串 S 中的位置。
- 如果当前的匹配位置距离主串 S 的最后字符的长度已经小于子串 T 的长度，则匹配失败。

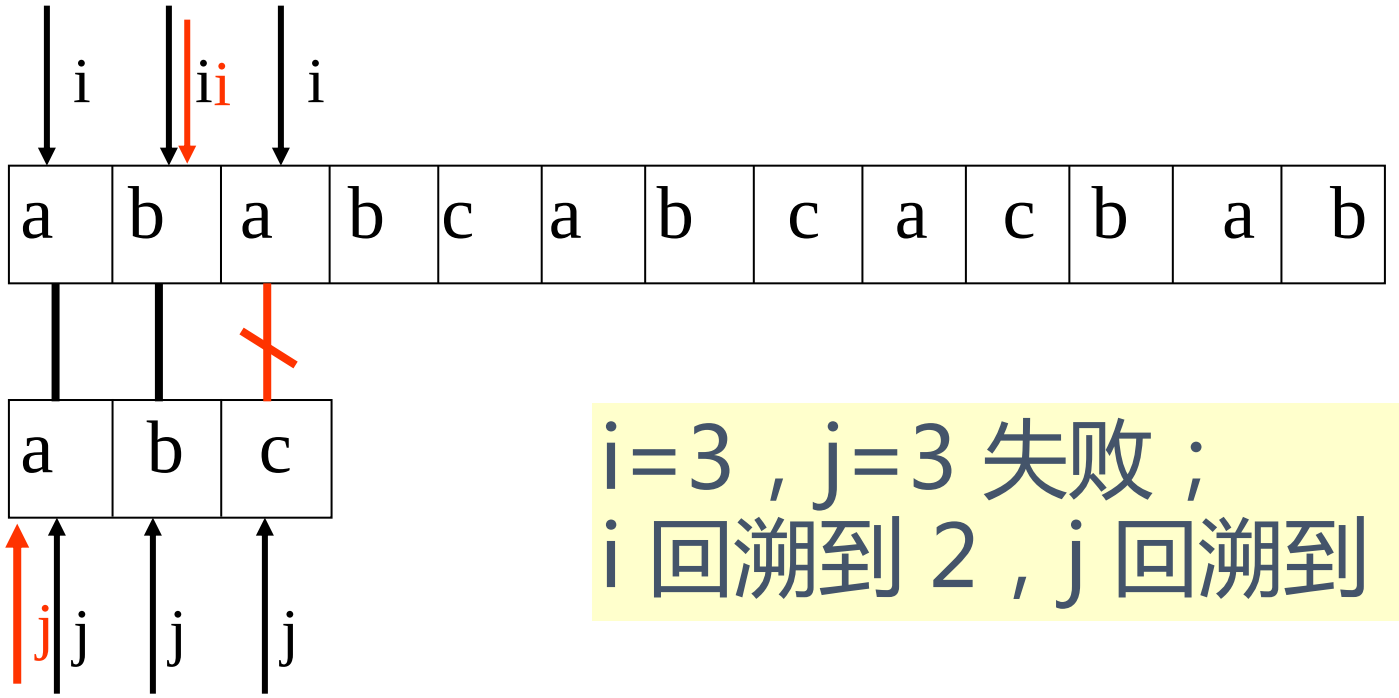
# 字符串匹配原理（续）


本趟匹配开始位置
回溯
i
主串S
$s_i$
……
i=i-j+2
成功
…
匹配失败
j=1
子串 T
$t_j$
j
回溯
i-j+2 即把上一次开始匹配时的位置加 1

# 字符串匹配例子

设主串 s=ababcabcacbab，子串 t=abcac

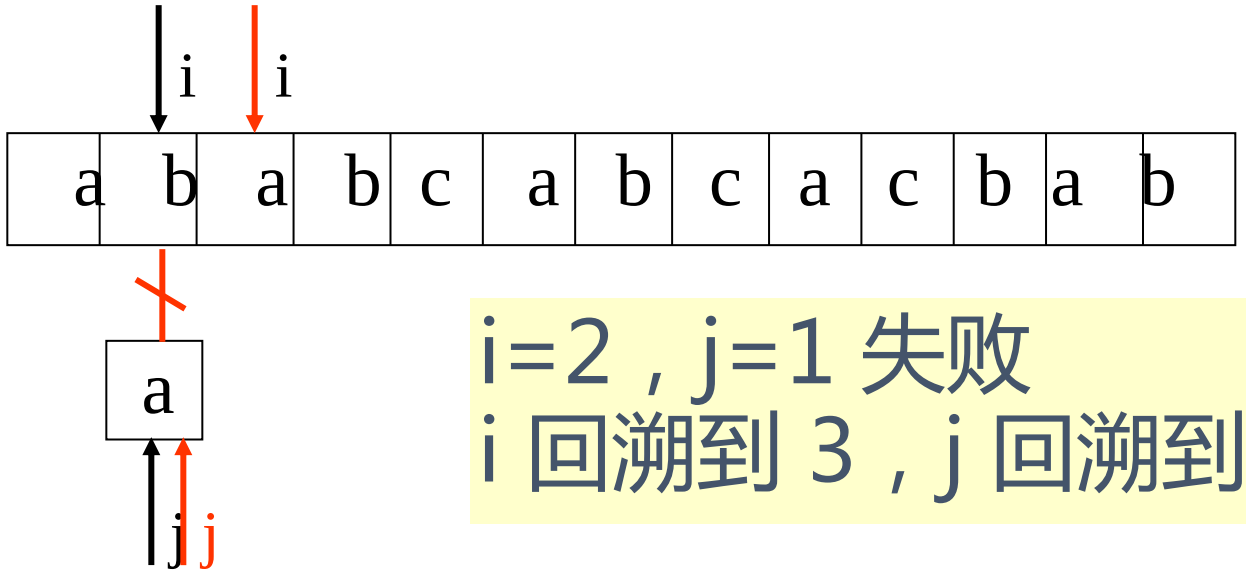
第1趟




i=3，j=3 失败；
i 回溯到 2，j 回溯到 1

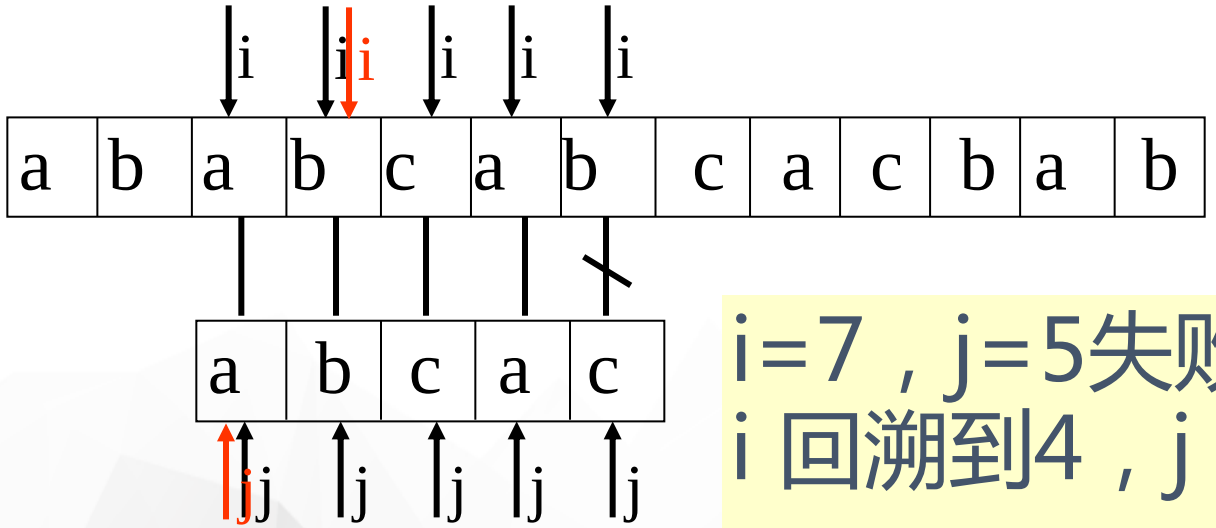
# 字符串匹配例子（续）

第2趟




i=2，j=1 失败
i 回溯到 3，j 回溯到 1

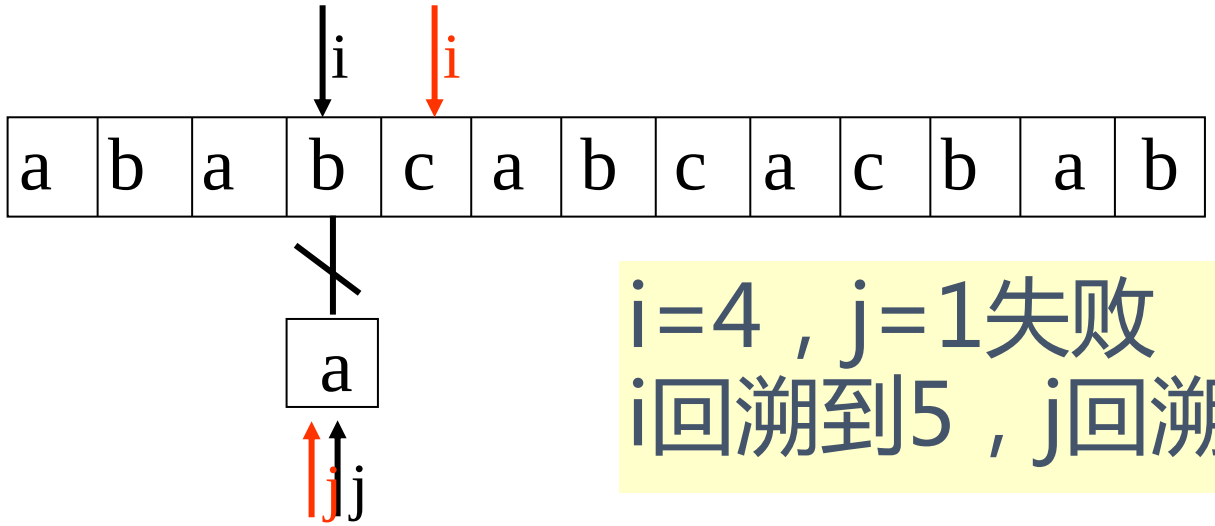
第3趟




i=7，j=5失败
i 回溯到4，j 回溯到1

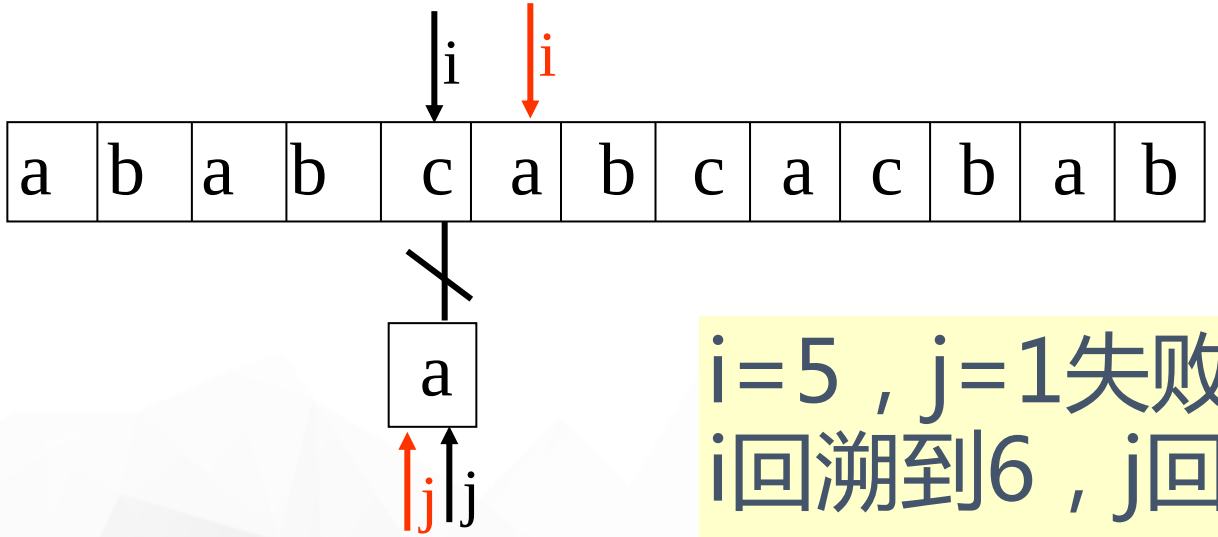
# 字符串匹配例子（续）

第4趟




i=4，j=1失败
i回溯到5，j回溯到1

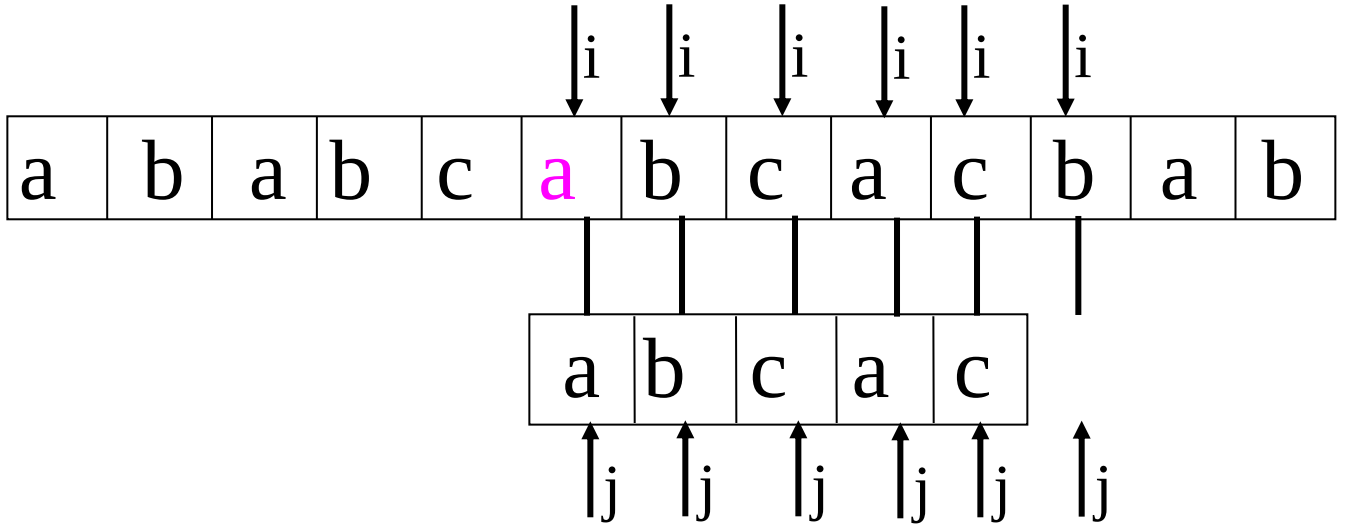
第5趟




i=5，j=1失败
i回溯到6，j回溯到1

# 字符串匹配例子（续）

第6趟




i=11，j=6，t 中全部字符都比较完毕，匹配成功。

# 字符串匹配蛮力算法1

```
int BF(char S[ ], char T[ ])
{
     int i=1; j=1;
     int n=strlen(S)        //主串长度
     int m=strlen(T)      //子串长度
    while((j<=m)  && (n-i+1>=m-j+1))
    {
        if (S[i]==T[j]) {i++; j++;}
        else { i=i-j+2;  j=1;  } //回溯
    }
    if (j>m) return (i-j+1);
    else return -1;          //匹配失败
}
```

# 字符串匹配蛮力算法2

```
int BruteforceIndex (W[0..n-1], M[0..m-1])
{
    for(int i=0; i<=n-m; i++)
    {
        int j=0;
        while (j<m)&&(W[i+j]==M[j])
        {
            j++;
            if (j==m) return i;
        }
    }
    return (-1);
}
```

每次匹配时 i 不变

匹配成功

匹配失败

# 字符串匹配算法的时间复杂性

- 1、算法输入规模的度量
  - n 和m，即文本和模式的长度。
- 2、算法的基本操作
  - W[i+j] 和 M[j] 之间的字符比较操作。
- 3、复杂性类型
  - 算法最内层循环存在return分支语句，循环的执行次数不只与输入规模有关，还与输入的具体情况有关，因此需要分析最好、最差和平均复杂度。

# 字符串匹配算法的时间复杂性（续）

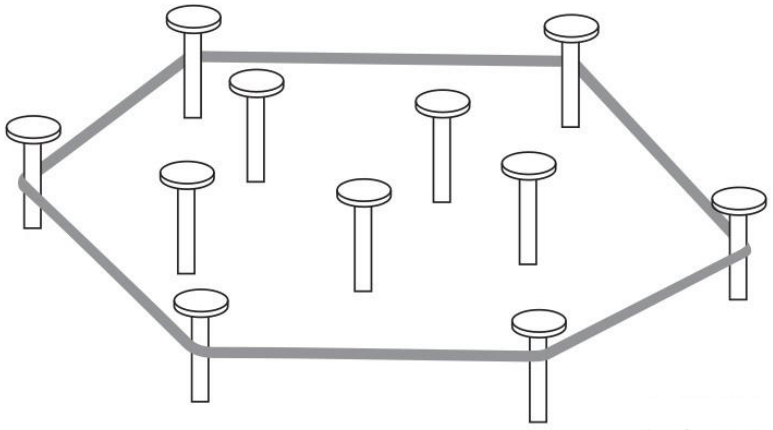
- 4、最好复杂度
  - 如果模式很短（m很小），则 $T_{best}(n,m)=m$。
    - 即在文本的第一个位置就匹配成功。
  - 如果文本和模式的长度相近（n-m很小），则 $T_{best}(n,m)=n-m$。
    - 即匹配失败，并且每次匹配均在第一个字符就匹配失败。
  - 综上 $T_{best}(n,m)=min(m, n-m)$ 。
- 5、最差复杂度
  - $T_{worst}(n,m) =(n-m+1)*m = O(nm)$。
    - 即外层循环执行n-m+1次，内层循环每次都执行m次。例如，主串为 aaaaaaaaaaaaaaaaac，子串为 aaab 时,会出现这种极端的情况。
  - 一般情况下这种情况很少出现，算法的平均效率要比最差效率好很多，大概是 O(n+m)的。

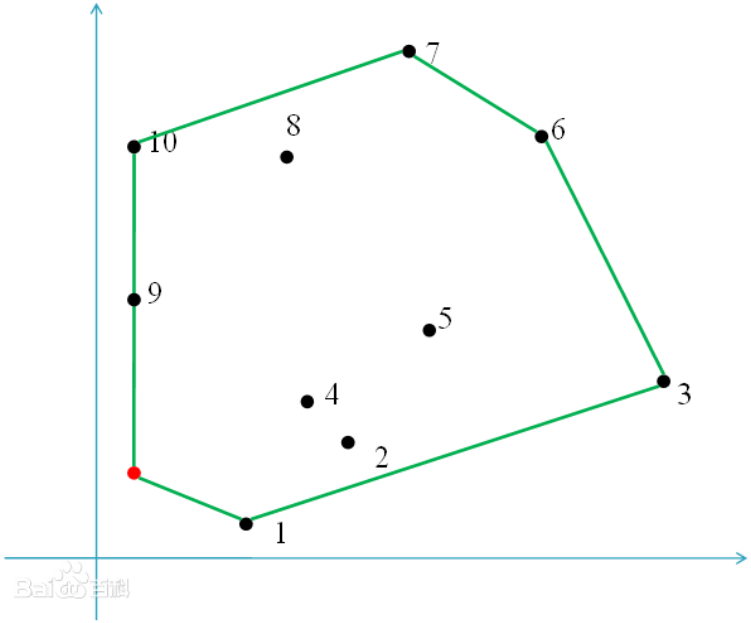
# 四、几何问题

- 4.1 凸包问题

# 4.1 凸包问题

- 凸包（Convex Hull）的定义
  - 凸包是包含平面上所有给定点的最小凸多边形（所有内角都小于180度）。
  - 凸包的任何一条边所在的直线把凸多边形全部划在了同一个半平面内。





# 凸包问题

- 特殊情况
  - 平面上只有两个点
    - 包含这两点的凸包是它们构成的线段。
  - 平面上有三个点
    - 如果这三点不共线，则包含这三点的凸包是以这三点为顶点构成的三角形。
    - 如果这三点共线，则包含这三点的凸包是由距离最远的两点构成的线段（第三点在线上）。
- 求凸包的常用算法有蛮力法、格雷厄姆扫描法、分治法 和 Jarris 步进法等。

# 凸包的性质和蛮力法的基本思想

- 凸包的性质
  - 如果点集中两个点的连线属于凸多边形的边，当且仅当点集中其余的点都在这两个点连线的同一侧。
- 蛮力法的基本思想
  - 对于由 n 个点构成的集合 S 中的两个点 $p_i$ 和 $p_j$，当且仅当该集合中的其他点都位于穿过这两点的直线的同一边时，它们的连线是该集合凸包边界的一部分。
  - 对集合 S 每一对顶点的连线都检验一遍，满足条件的线段构成了该凸包的边界。

# 具体方法

- 平面上穿过两个点($x_1$，$y_1$)和($x_2$，$y_2$)的直线由下面的方程定义：

$$ax十by=c$$

其中 $a=y_2-y_1, b=x_2-x_1, c=x_1y_2-y_1x_2$

- 该直线把平面分成了两个半平面
  - 其中一个半平面中的点都满足 ax+by > c。
  - 另一个半平面中的点都满足 ax十by < c。
- 为了检验其他点是否位于该直线的同一边，可以把每个点代入方程 ax十by，检验计算出的结果与 c 的大小关系是否相同。
  - 特殊情况：如果 ax+by=c，则说明第三点和前两点共线，这时需要重新审核可能的凸包边界。

# 蛮力法的时间复杂性

- 上述算法首先计算所有点两两组合的边，共有 n(n-1)/2 中情况。
- 每条边都要对其他 n-2个顶点求出在直线方程 ax+by=c中的符号，所以，其时间复杂度是$O(n^3)$。

# 五、组合问题

- 5.1 0/1背包问题
- 5.2 TSP问题
- 5.3 幂集问题

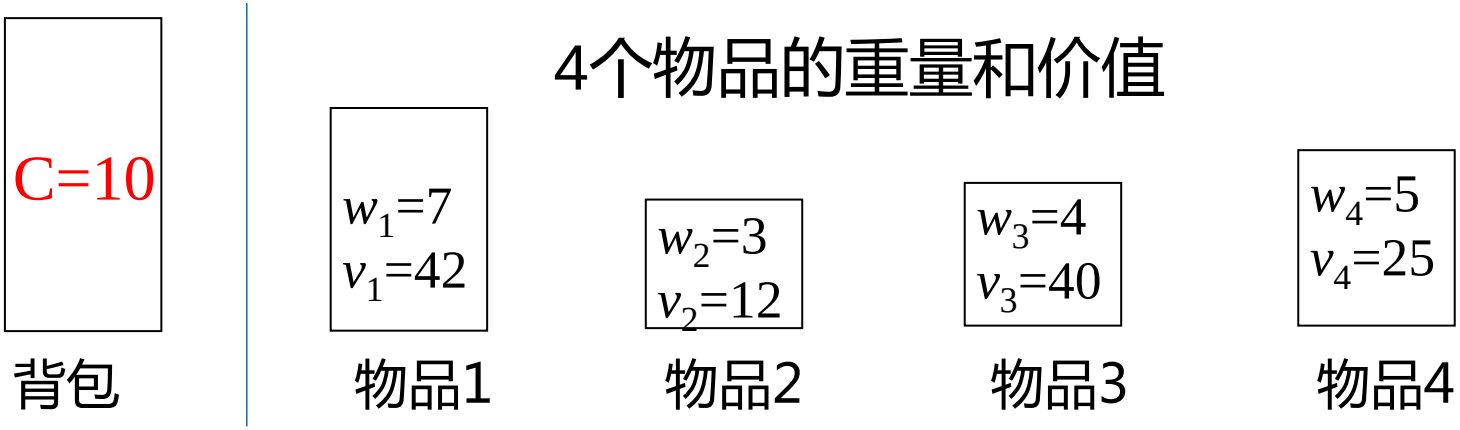
# 5.1 0/1背包问题

- 背包问题的名称来源于如何选择最合适的物品放置于给定背包中，在1978年由Merkel 和 Hellman 提出。
- 背包问题经常出现在商业、组合数学、计算复杂性理论、密码学和应用数学等领域中。
- 0/1背包问题是最基本的背包问题，指每个物品要么完全放入背包，要么完全不放入背包。

# 0/1背包问题描述

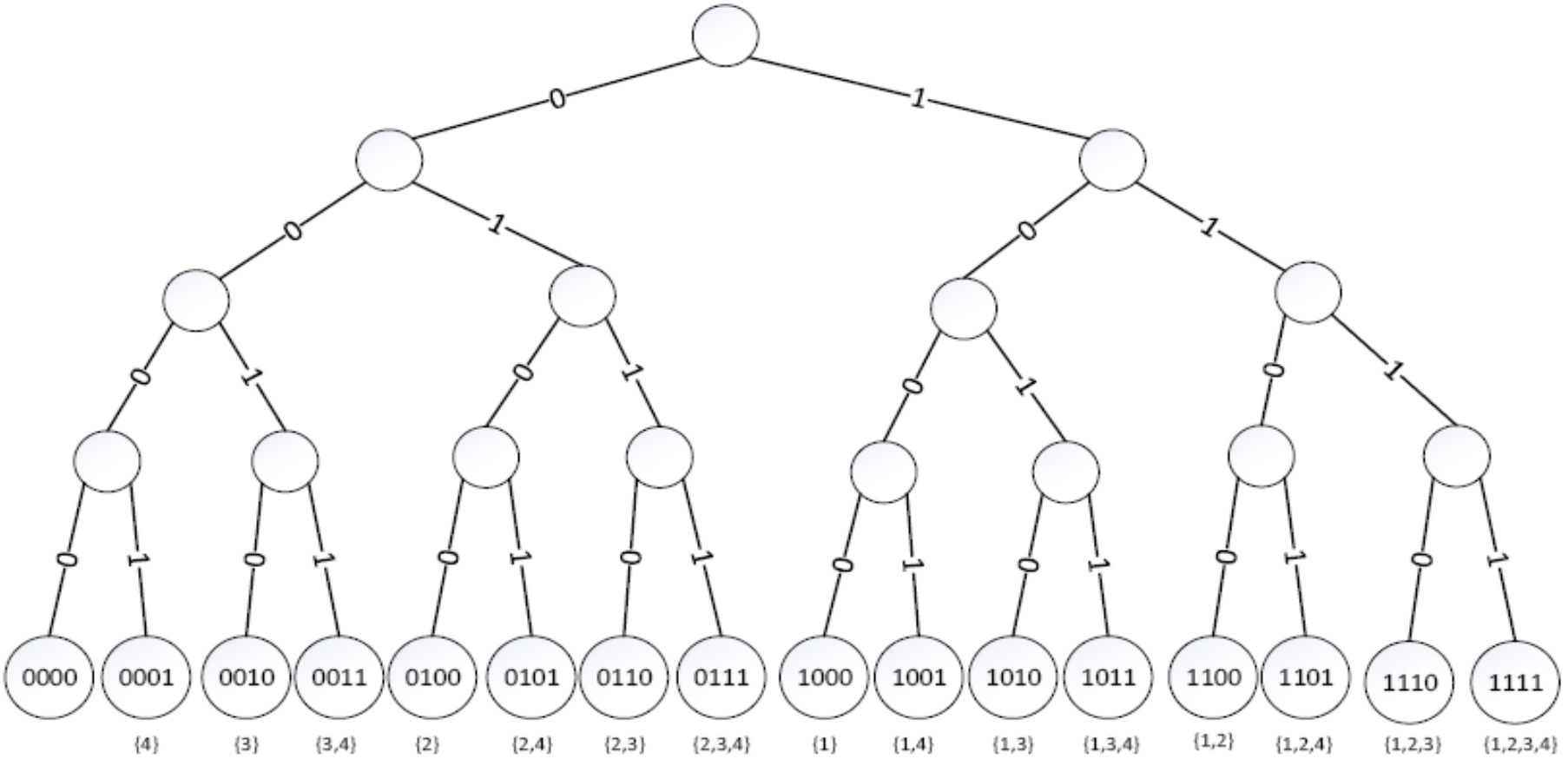
- 0/1背包问题描述
  - 给定 n 个重量为 $\{w_1, w_2, \ldots, w_n\}$、价值为 $\{v_1, v_2, \ldots, v_n\}$ 的物品和一个容量为 C 的背包，求这些物品的一个最有价值的子集，并且该子集要能够装到背包中。
- 蛮力法解决0/1背包问题的思路
  - 1、计算给定的 n 个物品的所有子集。
  - 2、找出总重量之和不超过背包容量的子集。
  - 3、计算上述每个子集的总价值，然后在它们中找到价值最大的子集。

# 0/1背包问题例子




| 序号 | 子集 | 总重量 | 总价值 | 序号 | 子集 | 总重量 | 总价值 |
|---|---|---|---|---|---|---|---|
| 1 | φ | 0 | 0 | 9 | {2,3} | 7 | 52 |
| 2 | {1} | 7 | 42 | 10 | {2,4} | 8 | 37 |
| 3 | {2} | 3 | 12 | 11 | {3,4} | 9 | ***65*** |
| 4 | {3} | 4 | 40 | 12 | {1,2,3} | 14 | 不可行 |
| 5 | {4} | 5 | 25 | 13 | {1,2,4} | 15 | 不可行 |
| 6 | {1,2} | 10 | 54 | 14 | {1,3,4} | 16 | 不可行 |
| 7 | {1,3} | 11 | 不可行 | 15 | {2,3,4} | 12 | 不可行 |
| 8 | {1,4} | 12 | 不可行 | 16 | {1,2,3,4} | 19 | 不可行 |

# 四个物品0/1背包问题的解空间树




子集树

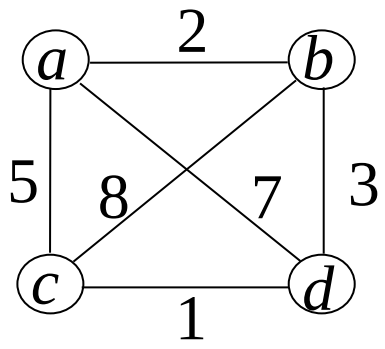
# 蛮力法求解0/1背包的复杂性

- 对于一个具有 n 个元素的集合，其子集数量是 $2^n$，所以，蛮力法求解 0/1 背包问题都会导致一个 $\Omega(2^n)$ 的算法。

# 4.2 TSP问题

- TSP（Travelling Salesman Problem）问题又称为货郎问题、邮递员问题，是图问题中最广为人知的问题。
- TSP问题的描述
  - 求旅行者由起点城市出发，通过所有给定的城市之后，最后再回到起点城市的最小路径成本。
- 用蛮力法解决TSP问题，要找出所有可能的旅行路线，从中选取路径长度最短的简单回路。

# TSP问题的例子



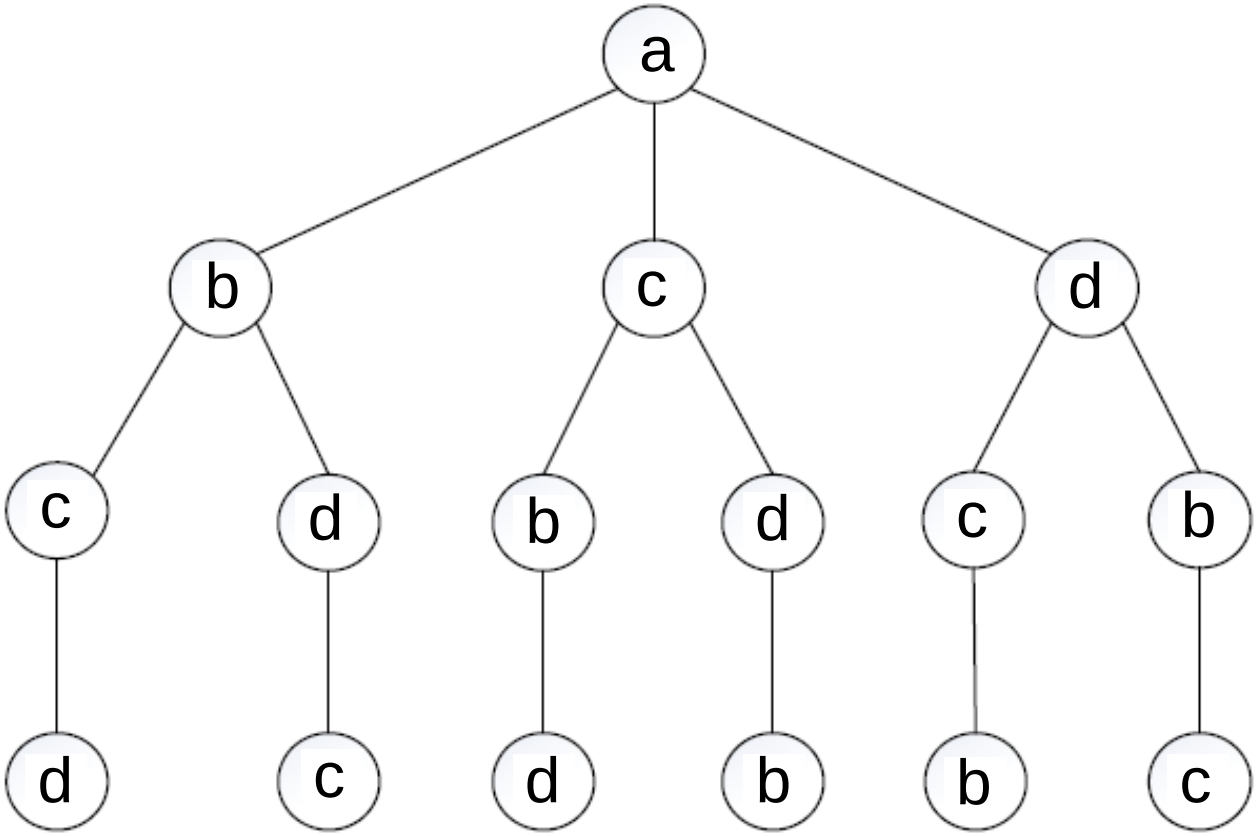

| 序号 | 路径 | 路径长度 | 是否最短 |
|---|---|---|---|
| 1 | $a\rightarrow b\rightarrow c\rightarrow d\rightarrow a$ | 18 | 否 |
| 2 | $a\rightarrow b\rightarrow d\rightarrow c\rightarrow a$ | 11 | 是 |
| 3 | $a\rightarrow c\rightarrow b\rightarrow d\rightarrow a$ | 23 | 否 |
| 4 | $a\rightarrow c\rightarrow d\rightarrow b\rightarrow a$ | 11 | 是 |
| 5 | $a\rightarrow d\rightarrow b\rightarrow c\rightarrow a$ | 23 | 否 |
| 6 | $a\rightarrow d\rightarrow c\rightarrow b\rightarrow a$ | 18 | 否 |
| … | … | … | … |

TSP问题的搜索空间：

从起点出发，遍历其余(n-1)个顶点的可能路径有（(n-1)！条。考虑到无向图中有一对路径只是方向不同，长度相同，因此可以将搜索空间减半，即可能的解有(n-1)!/2 个。

用蛮力法求解TSP问题，其时间复杂性为O(n!)，只能解决问题规模很小的实例。

# 四顶点图的TSP问题的解空间树




排列树

# 4.3 幂集问题

- 问题描述
  - 对于给定正整数n(n>=1)，求1-n构成的集合的幂集。
- 解法1
  - 直接穷举，将1-n存放到数组 a 中，求解问题变为构造集合 a 的所有子集合。下面是 n=3 的幂集：

| 集合元素 | 对应二进制位 | 对应十进制数 |
|---|---|---|
| {} | - | - |
| {1} | 001 | 1 |
| {2} | 010 | 2 |
| {1，2} | 011 | 3 |
| {3} | 100 | 4 |
| {1，3} | 101 | 5 |
| {2，3} | 110 | 6 |
| {1，2，3} | 111 | 7 |

# 求解幂集问题的算法

- 对于 n 个元素的集合 a，幂集求解如下：

```
for(i=0;i<2^n;i++)
{
    将 i 转换为二进制数 b；
    输出 b 中为 1 的位对应的 a 中元素构成一个集合元素；
}
```

# 求解幂集问题算法（续）

```
#define MaxN 10
void inc(int b[],int n)
{ for(int i=0;i<n;i++)
   { if (b[i])  b[i]=0;
      else
        {b[i]=1;
         break;}
   }
}
```

```
viod Pset(int a[],int b[],int n)
{   int i,k;
     int pw=(int)pow(2,n);
     printf("1-%d的幂集：\n",n);
     for(i=0;i<pw,i++)
     { printf("{");
       for(k=0;k<n;k++)
          if(b[k])  printf("%d",a[k]);
       printf("}");
       inc(b,n);
     }
     printf("\n");  }
```

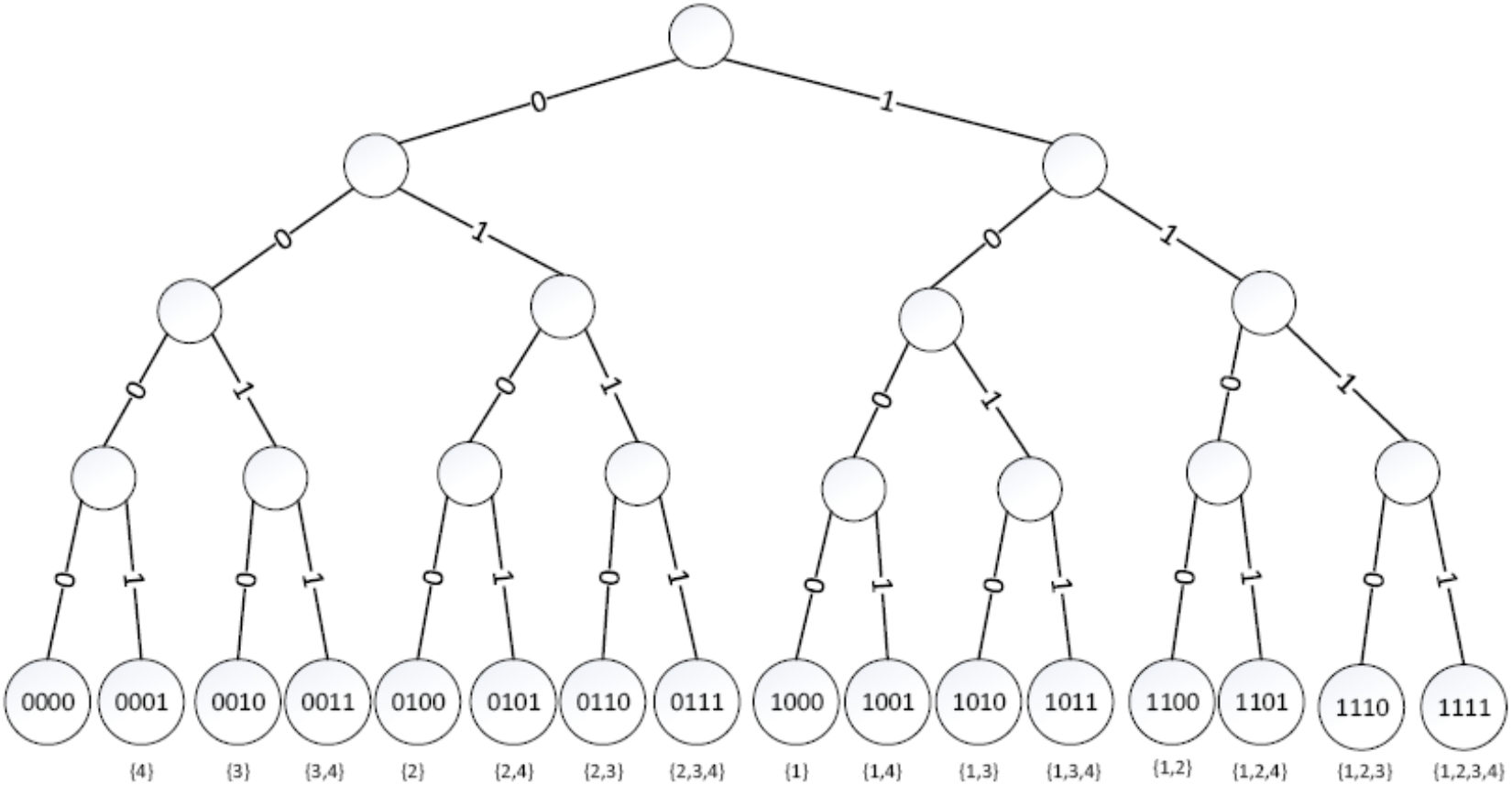
pw中循环$2^n$次，inc() 的时间为O(n)，所以算法时间复杂度为**$O(n*2^n)$**。

# 求解幂集问题算法（续）

- 解法2：采用增量穷举法
  - 1、产生一个空集元素{}添加到ps中，即ps={{}}。
  - 2、在步骤1得到的ps的每一个集合元素的末尾添加1构成新集合元素{1}，将其添加到ps中，即ps={{},{1}}。
  - 3、在步骤 2 得到的每一个集合元素的末尾添加 2 构成新集合元素{2}，{1,2}，将其添加到ps中，即ps={{},{1} {2},{1,2}}。
  - 4、在步骤 3 得到的每一个集合元素的末尾添加 3 构成新集合元素 {3},{1,3},{2,3},{1,2,3}，将其添加到ps中，即ps={{},{1} {2},{1,2}, {3}, {1,3},{2,3},{1,2,3}}。
  - 最后的 ps 构成 {1, 2, 3} 的幂集。
- 时间复杂度：$O(2^n)$

# 求解幂集问题算法（续）

- 解法3：通过递归构建子集树




4个元素的子集树

# 本章小结

- 一、蛮力法概述
- 二、排序问题
- 三、查找问题
- 四、几何问题
- 五、组合问题